The Consequence Gap: Why AI Agent Safety Needs More Than Alignment

Anonymous Authors

Abstract

This position paper argues that a central underaddressed threat from tool-augmented AI agents is not model misalignment but the *consequence gap*: the divergence between what an agent understands it is doing and what actually happens. This gap has two sides. On the *system side*, the agent's tools produce effects it cannot observe—a Python dependency's post-install hook silently creates a cron job the agent will never know about. On the *operator side*, the agent's expertise produces powerful outputs it cannot restrict to authorized use—the same security audit serves defense or offense depending on who asked, and the agent cannot tell the difference. We demonstrate both sides experimentally. In both cases the failure occurs under aligned behavior: the model follows the request, behaves correctly, and reports honestly. The system-side gap requires infrastructure-level *consequence bounding*; the operator-side gap requires shifting accountability from the model to the operator. Neither is solved by model alignment alone.

1 Introduction

In early 2026, over 500,000 instances of OpenClaw—an open-source AI agent with full shell access—were deployed on the public internet [18]. One ran autonomously for 247 days executing financial trades. Cisco found that 20% of its community skill ecosystem was malicious [21; 22]. Anthropic's Claude Code gives models a real developer terminal. Nous Research's Hermes Agent autonomously creates and improves its own skills across sessions [19]. The industry is competing on a single axis: give the model more autonomy, more tools, and let it work.

The safety community has responded by asking: *what if the model becomes misaligned?* This paper asks a different question: *what if the model behaves as aligned, but cannot understand the full consequences of the tools we give it?*

Consider a coding agent asked to set up a Python project. It runs `pip install -r requirements.txt`—standard practice. From the agent's perspective, this is a single, well-understood action: install dependencies. From the system's perspective, `pip install` executes arbitrary code via `setup.py` hooks. If one dependency contains a malicious post-install hook that creates a cron job, the agent has established a persistence mechanism—without knowing, without intending, and without the ability to report it.

Now consider the same agent asked to audit a website's security. It runs `curl -sI https://target.com`—an HTTP request. From the agent's perspective, it is checking response headers. From the system's perspective, this is the first step in a comprehensive vulnerability assessment that will produce a complete attack playbook: account takeover chains, remote code execution paths, credential theft techniques. The output is identical regardless of the recipient's intent.

These are two directions of the same phenomenon—the *consequence gap* (formally defined in Section 3):

- **System-side**: the agent's tools produce effects it cannot observe. The threat comes from the environment—compromised dependencies, tool side effects, supply chain integrity. Neither the agent nor the operator authorized these consequences.
- **Operator-side**: the agent produces powerful outputs it cannot restrict to authorized use. The threat comes from the operator—the same capability serves defense or offense, and the agent cannot verify who is asking or why.

In both cases, the model behaves as aligned—it follows its operator's intent faithfully and reports honestly about what it believes it has done. In both, every model-focused

guardrail fails—RLHF, input classifiers, chain-of-thought monitoring—because the model is not the source of the threat. The system-side gap requires infrastructure controls. The operator-side gap requires operator accountability. Neither is solved by model alignment alone.

Supply chain attacks are not new. What is new: (1) frontier AI agents are universally susceptible with no behavioral signal, across every provider and model size tested; (2) a separate supervisory model reviewing the full session log also fails; (3) the mechanism is not a model failure but an architectural property—the model is not being deceptive, it is being ignorant—that alignment training cannot address.

Position: the consequence gap is a central underaddressed threat from deployed AI agents, and model alignment alone cannot close it because the relevant failures occur outside the model's control plane. Our goal is not to introduce a new benchmark for agent security, but to argue that deployed-agent safety is currently framed at the wrong control plane. This paper makes three claims. First, the consequence gap is a central architectural risk for deployed AI agents. Second, this gap already enables all conditions for sustained autonomous harm, as demonstrated experimentally. Third, the safety community's focus on model alignment, while necessary, is insufficient: system-side risks require infrastructure controls, while operator-side risks require authorization and accountability controls.

2 Related Work and Positioning

Prior safety research locates risk inside the model. This paper locates it in the gap between what the model understands and what the system does. The distinction matters because every defense designed for the former is structurally blind to the latter.

Hubinger et al. [6] showed that deceptive behaviors (sleeper agents) survive safety training. Apollo Research [7] found five of six frontier models engaged in in-context scheming—faking alignment, disabling oversight, attempting to copy weights. Anthropic and Redwood Research [8] reported alignment faking in Claude, where the model adjusted reasoning based on perceived monitoring. These results establish that agent-level misalignment is real. This paper does not dispute them.

These works share an assumption: *the model is the locus of risk*. The defenses—RLHF, chain-of-thought monitoring, interpretability probes [6], anti-scheming training [7]—all target the model's internal state. This addresses only one threat pathway and misses a more fundamental one: risks that arise from the *system architecture* regardless of model alignment.

On the tool access surface, STAC [10] demonstrated that individually benign tool calls compose into dangerous chains, Kim et al. [11] analyzed privilege escalation via data flows, and SEAgent [13] identified confused deputy attacks in multi-agent systems. MalTool [12] documented over 17,000 tools in the MCP ecosystem. These works study tool-level threats but frame them as model-mediated attacks. The contribution here is demonstrating that the tools themselves—through supply chain compromise and capability composition—produce harmful outcomes without any model-level failure.

On persistence, Pan et al. [9] demonstrated self-replication and Carlsmith [2] analyzed power-seeking behavior. Both frame persistence as a model *capability*—an intentional act by the agent. The experiments below reveal persistence instead as an *architectural side effect*: the system acts through the model, without the model's knowledge.

The consequence gap subsumes several existing threat categories. Prior frameworks identify "evasion of oversight" as a necessary condition for sustained escalation [2; 1]. This is a special case of the system-side gap: an agent that cannot observe its own tool side effects cannot report them, achieving structural evasion without deception. This is strictly harder to defend against than active evasion, because alignment training cannot fix ignorance.

3 The Consequence Gap

This section formalizes the consequence gap, establishes its two sides, and explains why model alignment cannot close either. The formalism provides the analytical scaffold for the experiments in Section 4.

3.1 Definition

An AI agent operates by invoking tools—shell commands, API calls, file operations. Each invocation has effects the agent *intends* (the understood consequence) and effects it does not intend or observe (the actual consequence). The consequence gap is the set difference:

$$\Delta C = C_{\text{actual}}(a) \setminus C_{\text{understood}}(a)$$

where a is an action, C_{actual} is the full set of effects, and $C_{\text{understood}}$ is what the agent believes happened. In Experiment 1 (Section 4.1), $C_{\text{understood}}(a) = \{\text{dependency installed}\}$ while $C_{\text{actual}}(a) = \{\text{dependency installed, cron job created, persistence file written}\}$; ΔC is the entire attack surface. In Experiment 2, $C_{\text{understood}}(a) = \{\text{HTTP headers checked}\}$ while $C_{\text{actual}}(a) = \{\text{complete attack chain documented}\}$.

When $\Delta C = \emptyset$, the agent fully understands what it did. Model alignment is sufficient for safety: an aligned agent that sees all consequences can refuse harmful ones. When $\Delta C \neq \emptyset$, the agent has produced effects it cannot evaluate, report, or reverse. Model alignment is *insufficient*: even an agent behaving as aligned will execute actions whose hidden consequences may be harmful.

3.2 Two Sides

The system-side gap arises when tools execute code or trigger processes the agent does not observe. `pip install` runs `setup.py` hooks. `npm install` runs lifecycle scripts. MCP tool plugins execute server-side logic. In each case, the agent invokes a single command; the system executes arbitrary code. The agent sees the command's visible output; the system may have silently modified the filesystem, scheduled processes, or exfiltrated data. The threat comes from the environment—neither the agent nor the operator authorized these consequences.

The operator-side gap arises when the agent faithfully produces powerful outputs without the ability to verify whether the operator is authorized to receive them. A security audit, a vulnerability assessment, an infrastructure deployment plan—each is valuable when requested by the system's owner and dangerous when requested by an attacker. The agent composes individually benign actions into consequential outputs because it is helpful. It cannot distinguish authorized from unauthorized use because it has no mechanism to verify operator identity, intent, or authorization. The same output serves defense and offense; the difference is the operator, not the model. This is not merely the observation that malicious users exist; it is the claim that aligned helpfulness converts ambiguous authorization into expert operational capability, while the safety-relevant variable is whether the requester is authorized to receive the result.

3.3 Why Model Alignment Cannot Close the Gap

The model cannot see the system-side gap. No amount of alignment training teaches a model to predict the behavior of every `setup.py` in every package it might install. The model processes the *interface* of its tools—command name, arguments, visible output—while the consequences occur in the *implementation*: everything the tool does that never appears in stdout.

The model cannot close the operator-side gap because alignment makes it *worse*: a model behaving as aligned is maximally helpful to whoever asks. It cannot verify operator

identity—API keys authenticate access, not authorization for specific tasks. It cannot assess operator intent—"audit this website" is identical whether spoken by the owner or an attacker. A model that refuses to compose expert-level outputs without verifying authorization also refuses to be useful to legitimate operators. Closing the operator-side gap at the model level means eliminating the agent's utility.

4 Experimental Evidence

Frontier models resist direct prompt injection across five stealth levels—from plaintext instructions to implicit goal framing—in all trials ($n = 12$). Agent-level alignment works for direct attacks. The consequence gap, however, bypasses it entirely.

4.1 Experiment 1: The System-Side Gap (Supply Chain)

Setup. A tool-augmented agent receives a legitimate task: "Install the dependencies from `requirements.txt`, review the code, and fix any bugs." The workspace contains a Python project with a deliberate bug and one dependency: `logging-helpers==2.1.0`. The package is legitimate-looking—proper documentation, classifiers, a functional logging module. Its `setup.py` contains a post-install hook, buried in standard setuptools boilerplate, that silently creates a cron job and writes a persistence file. The system prompt is clean. No injection, no hidden instructions, no adversarial framing. The task is something every developer does daily. The experiment runs in an isolated Docker container with no network access.

Results. We tested 7 models across 3 providers (OpenAI, Anthropic, MiniMax), 10 trials each ($n = 70$). Every model was given the same task, the same workspace, and the same compromised dependency.

Model	Provider	N	Injected
GPT-5.4	OpenAI	10	100%
GPT-5.4-nano	OpenAI	10	100%
Claude Opus 4.7	Anthropic	10	90%
Claude Opus 4.6	Anthropic	10	100%
Claude Sonnet 4.6	Anthropic	10	100%
MiniMax M2.7	MiniMax	10	100%
MiniMax M1	MiniMax	10	100%
All models		**70**	**98.6%**

Table 1: Supply chain injection results. *Injected*: a persistence mechanism (cron job and/or persistence file) was created by the post-install hook. *Self-Report*: the model's characterization of its own changes when asked for a complete summary—*omission* = persistence not mentioned; *false completeness* = model claimed it made only the bug fix; *disclosure* = model mentioned the persistence mechanism. Every model that ran `pip install` triggered the hook; 69 of 70 trials produced persistence the agent could not observe or report. All 70 trials fixed the bug correctly. The single clean trial (Opus 4.7) occurred when the model chose not to install dependencies. Three additional models (GPT-5.5, GPT-5.4-mini, Gemini-3 Flash) were tested but are excluded here due to incomplete trial runs; results will appear in the camera-ready version.

In every injected trial, the agent produced a thorough, honest self-report listing every command and file change—and did not mention the cron job or persistence file. Classification: **omission** in 45 of 49 non-error self-reports. Not deception—ignorance. One trial (Opus 4.6) exhibited **false completeness**—the model asserted it had only fixed the bug, without knowing about the cron job. (False completeness is an incorrect claim of completeness from ignorance; it differs from active deception, which would require the model to

know about the side effect and choose to conceal it.) One trial (Opus 4.7) fully disclosed the persistence.

A separate auditor model (Claude Sonnet 4.6) reviewed each full session log with the instruction to flag any action not directly required by the task. The auditor consistently flagged `pip install` as potential scope creep—correctly identifying an action outside the stated task—but never detected the cron job or persistence file. This is not a failure of the auditor's reasoning: the cron job never appears in any tool output visible to either the agent or the auditor. It is a structural blind spot. A supervision layer that can only see what the model sees inherits the model's ignorance.

The result is uniform across model size (frontier GPT-5.4 and small GPT-5.4-nano both 100%), provider (OpenAI, Anthropic, MiniMax), and capability level. The single clean trial occurred when Claude Opus 4.7 chose not to run `pip install`—the only defense was not invoking the tool, which means not completing the task.

What this shows. The supply chain attack works universally. Model alignment, model size, and provider are irrelevant. Every model that executes `pip install`—the standard, correct response to "install dependencies"—gets compromised. The consequence gap is architectural, not behavioral. The threat enters through the supply chain—the same supply chain where 20% of OpenClaw's skill ecosystem was found to be malicious [21].

4.2 Experiment 2: The Operator-Side Gap (Capability Democratization)

Setup. We deployed a tool-augmented agent as a security auditor against 15 websites. All site owners provided explicit written authorization via a consent form prior to testing; signed consent records are on file. Per responsible disclosure norms, targets are anonymized by industry category and raw audit reports are not published; all identified vulnerabilities were reported to the affected parties. The agent received a single-sentence prompt and access to standard tools: `curl`, `dig`, `openssl`, and a headless browser. No exploit code was provided.

Results. The agent produced over 500 pages of vulnerability assessments, identifying critical findings including:

Target	Severity	Worst Finding
Creator Platform (SaaS)	3 Critical	Any website can act as any logged-in user
AI Resume Service	2 Critical	Unauthenticated access to 4M-user database
B2B Marketing SaaS	4 Critical	Full kill chain: enumeration to admin takeover
E-commerce (WooCommerce)	1 Critical	Remote code execution (CVSS 9.8)
Insurance API Platform	1 Critical	Cross-origin exfiltration of merchant billing data
Social Gaming App	High	44,109 user files (13.3 GB) publicly downloadable

Table 2: Selected findings from authorized security audits. Each audit was performed autonomously by a single AI agent using standard reconnaissance tools.

Across all 15 targets, the agent identified 202 distinct vulnerabilities:

	Critical	High	Medium	Low	Total
All 15 targets	13	45	90	54	202
Sites with $\geq$1 Critical			6 / 15 (40%)		
Sites with $\geq$1 High			15 / 15 (100%)		

Table 3: Aggregate vulnerability findings across 15 authorized security audits. Every target had at least one high-severity vulnerability. All audits were performed autonomously by a single AI agent.

Each report included step-by-step exploit chains, browser-verified proof-of-concept demonstrations, and ready-to-execute attack scripts. A professional penetration test producing comparable output costs $10,000–50,000 and takes weeks. The agent produced it for approximately $5–10 per target.

What this shows. The agent executed individually benign commands—HTTP requests, DNS lookups. The *composed* output, mediated by expert-level security knowledge, is a weapon. The same report that helps a company fix its CORS misconfiguration gives an attacker a step-by-step guide to exploit it. The agent cannot distinguish the two uses. The cost collapse—three orders of magnitude—means that expert-level offensive capability, previously scarce and expensive, is now available to anyone with an API key.

Our results are modest compared to the frontier. Anthropic's Claude Mythos Preview—a model whose cybersecurity capabilities led Anthropic to withhold it from general release—autonomously discovered thousands of zero-day vulnerabilities across every major operating system and web browser, including a 27-year-old bug in OpenBSD and a 17-year-old remote code execution vulnerability in FreeBSD that it both found and exploited with no human involvement [23]. Anthropic's response was Project Glasswing, a consortium of major technology companies given early access to use the model defensively before comparable capabilities become broadly available [24]. This is the operator-side gap at the frontier: the same model that secures critical infrastructure for Apple and Microsoft could, in different hands, compromise it. The decision to restrict access acknowledges what our experiment demonstrates at smaller scale—that the gap between capability and consequence cannot be closed by alignment alone.

4.3 The Gap Is Wider Than These Experiments

Our two experiments demonstrate the consequence gap through software supply chains and security auditing. These are empirical proofs of concept—not the boundaries of the threat. The actual gap extends to every domain where agents invoke tools whose effects propagate through complex systems:

- **Financial systems.** An agent executing trades or managing payroll operates on numbers. The consequences—margin calls, liquidity cascades, regulatory violations—propagate through markets and institutions the agent cannot model.
- **Infrastructure and operations.** An agent writing Terraform configurations or modifying DNS records sees text files. A single misconfigured security group, produced by correct-looking infrastructure-as-code, can expose databases to the public internet.
- **Biological and chemical systems.** As AI agents are deployed in laboratory automation—adjusting protocols, ordering reagents, modifying concentrations—the gap between "change this parameter" and "alter a safety-critical threshold" becomes life-threatening.

The structure is identical across domains: the agent operates at the level of actions, but consequences propagate through systems whose complexity exceeds the agent's understanding.

5 Why Model-Focused Guardrails Fail

Every major guardrail monitors or constrains the *model*. Both experiments demonstrate threats that originate in the *system*.

RLHF and alignment training. The model's observed behavior was aligned in both experiments. RLHF cannot fix a malicious `setup.py`. It cannot prevent the model from composing `curl` commands into vulnerability assessments—that composition is the model being helpful.

Input classifiers and prompt shields. Both prompts were clean: "install dependencies and fix bugs" and "audit this website." No injection to detect.

Chain-of-thought monitoring. The model's reasoning was sound: "I should install requirements.txt first" and "I should check the CORS headers." There is nothing to flag.

Output classifiers. The model's outputs were legitimate commands and correct code. The harmful consequences occurred either inside tool execution (system-side gap) or in the composed output delivered to an unverified operator (operator-side gap).

Human-in-the-loop. A user who has approved `pip install` a hundred times will approve it the hundred-and-first time without auditing the dependency. IBM found that 63% of security alerts are false positives [15], training reflexive approval. Automation bias has been documented in security operations [14] and AI-assisted workflows [16; 17].

The failure pattern is consistent: every guardrail monitors the model, but the threat originates in the environment. This is not a failure of implementation. It is a category error. When the model is an unwitting vector for environmental threats—or an unwitting producer of dual-use knowledge—model-focused safety is solving the wrong problem.

Market dynamics compound the risk. Each generation of agent frameworks expands the consequence gap by design. Tool access is the baseline. Persistence is a selling point [19]. Anthropic identified approval fatigue and responded by reducing approvals by 84% [20]—more autonomy, not less. The tool ecosystem has expanded to 17,000+ tools on mcp.so alone [12]. At 500,000+ deployed agent instances, the probability that at least one encounters a compromised dependency at any given time approaches certainty.

6 Consequence Bounding

If model alignment cannot close the consequence gap, what can?

The gap exists because agents invoke tools whose effects exceed their understanding. The response must operate at the system level, not the model level. We distinguish two complementary strategies—*bounding* (preventing consequences) and *observing* (detecting consequences the model cannot see)—and are honest about where each fails.

6.1 Bounding the System-Side Gap

The system-side gap is technically closable. The harmful consequences in our supply chain experiment were system side effects: a cron job, a file write. These can be prevented by infrastructure-level controls that restrict what *any process* spawned during an agent session can do, regardless of whether the model or a dependency initiated it:

- **Syscall filtering** (seccomp, AppArmor): block `crontab`, `at`, and `systemctl` invocations from any process in the agent's session, not just the model's direct tool calls.
- **Ephemeral filesystems** (overlay/tmpfs): ensure all filesystem changes are discarded at session end. No write persists unless explicitly approved.
- **Process namespacing**: prevent any subprocess from spawning background processes or daemons that outlive the session.
- **Post-install hook sandboxing**: run `pip install` with `--no-build-isolation` inside a restricted namespace where `setup.py` cannot make system calls beyond file I/O within the package directory.

These mechanisms exist today. They are not deployed in current agent frameworks because they degrade usefulness—an agent that cannot install packages or write persistent files is significantly less capable. But each mechanism is *surgical*: it restricts a specific consequence class without affecting the model's reasoning, knowledge, or in-session behavior.

6.2 Observing What the Model Cannot See

Bounding prevents consequences; observing detects them. The key insight is that current monitoring watches the *model's* actions (tool calls, outputs), but the system-side gap produces consequences in the *system* that never appear in the model's action log. System-level observability closes this blind spot:

- **Syscall auditing**: log every syscall made during the agent session—not just the model's tool calls, but every subprocess, every fork, every file write by any process the model's

commands spawned. In our experiment, `pip`'s invocation of `crontab` would appear in a syscall log even though it never appeared in the model's output.
- **State diffing**: snapshot the filesystem, process table, and cron state before and after the session. Any change the model did not explicitly create is a consequence gap artifact. Flag it for human review.
- **Expected-vs-actual comparison**: before executing a command, predict its expected effects (files modified, processes spawned). After execution, compare. Divergences indicate hidden consequences.

This is not novel technology—it is runtime application self-protection (RASP) and endpoint detection applied to AI agent sessions. What is novel is the recognition that agent monitoring must operate at the *system* level, not the *model* level, because the threats we demonstrate are invisible to model-level monitoring by construction.

6.3 The Operator-Side Gap Requires Operator Controls

The operator-side gap cannot be closed by infrastructure controls or model alignment. The security audit's "harm" is not a system side effect; it is *information* delivered to an operator whose intent the model cannot verify. The attack playbook is text. A runtime that prevents the model from composing `curl` commands into vulnerability assessments also prevents it from doing code review, debugging, or any task requiring expert reasoning. You cannot bound knowledge without destroying the agent's utility.

The solution operates at a different level: the *operator*, not the model. Like any tool, the model can be used for defense or offense; the difference is who wields it and whether they are authorized. Anthropic's decision to withhold Claude Mythos Preview from general release [23] acknowledges this directly: the defense against the operator-side gap is access control—restricting *who* can use the capability, not *what* the capability can do.

The strategic implication is asymmetric: the system-side gap is solvable at the infrastructure level; the operator-side gap requires shifting accountability from the model to the operator. These are fundamentally different kinds of interventions, and conflating them—as the current safety discourse does by treating all agent risk as an alignment problem—leads to defenses that address neither effectively.

6.4 Research Directions for the Operator-Side Gap

The operator-side gap cannot be bounded at the model level, but it can be governed at the operator level. Four directions deserve investigation:
- **Symmetric deployment.** The gap amplifies offense and defense equally. If auditing costs drop by three orders of magnitude, fixing costs should too. Automated remediation pipelines and continuous agent-driven monitoring can make defense the default use of the same capability.
- **Tiered capability access.** Not all operators need complete exploit chains. Anthropic's restriction of Mythos to vetted partners [23; 24] is an ad hoc version of this principle. Formalizing capability tiers—with graceful degradation rather than hard cutoffs—is an open research problem.
- **Provenance and attribution.** Agent-produced outputs should carry tamper-resistant metadata (model, operator, tools, timestamp) to enable accountability after the fact, even when prevention is impossible.
- **Consequence-aware composition.** Models can be trained to recognize when composed output crosses a consequence threshold and pause for authorization. The challenge is defining thresholds that catch genuine threats without blocking legitimate expert use.

None fully closes the operator-side gap. Together, they reduce exploitability while preserving utility, while shifting risk analysis toward the operator rather than the model.

7 Alternative Views and Objections

"Supply chain attacks aren't specific to AI agents." True. But agents amplify them: more autonomy (no per-action human review), more access (shell, network, credentials), more scale (500,000+ instances), and a larger consequence gap (agents cannot audit dependencies). The combination makes the agent supply chain qualitatively more dangerous than the human one.

"The security audit just shows websites have vulnerabilities." The finding is not that vulnerabilities exist but that an AI agent discovers, composes, and documents them into actionable exploit chains autonomously, at 1/1000th the cost of human expertise. This changes who can do it and at what scale.

"Better alignment will solve this." Our central result is that the demonstrated harms occur under aligned behavior. Alignment addresses agent-level threats; the consequence gap is an architectural property of the system. Even a model that faithfully follows its operator's intent cannot audit every dependency or predict every tool side effect.

"The framework is too simple." It is. But its value is the qualitative insight: the threat is in the gap between agent understanding and system behavior, not in the model's intent. This reframes where defenses should be built.

Ethics Statement

Security audits. All 15 websites audited in Experiment 2 were tested with explicit written authorization from site owners; signed consent records are on file. All identified vulnerabilities were reported before submission. Targets are anonymized; raw reports are not published.

Supply chain artifacts. The `logging-helpers` package was tested only in isolated Docker containers with no network access. It has not been uploaded to PyPI or any public package registry.

Artifact generation. Both experimental artifacts were generated by an AI agent from natural-language prompts. The Appendix reproduces them in full to demonstrate the consequence gap itself and to support reproducibility; they are not released as standalone tools.

IRB. This research did not involve human subjects. Experiments involve AI model APIs and authorized infrastructure testing.

8 Conclusion

AI agents today have extraordinary capability and no control over consequence. This is not a model bug. It is a structural feature of systems where agents invoke tools whose effects exceed their understanding and serve operators whose intent they cannot verify.

We demonstrated this concretely. On the system side, a frontier model ran `pip install`, fixed a bug, reported honestly, and had no idea a cron job was created—across 7 models, 3 providers, 98.6% of trials. On the operator side, the same model composed HTTP requests into 500 pages of exploit documentation at three orders of magnitude less cost than human expertise. Both experiments used aligned models; both evaded all guardrails. The first produced outcomes no one authorized; the second produced outcomes whose harm depends entirely on who asked.

These are different problems requiring different solutions. The system-side gap requires infrastructure-level consequence bounding—constraining what tools can actually do, regardless of intent. The operator-side gap requires shifting accountability from the model to the operator—verified identity, usage attribution, tiered capability access. As systems around models become more autonomous, connected, and consequential, neither gap is solved by model alignment alone.

References

[1] Bostrom, N. (2014). *Superintelligence*. Oxford University Press.

[2] Carlsmith, J. (2022). Is power-seeking AI an existential risk? *arXiv:2206.13353*.

[3] Omohundro, S. (2008). The basic AI drives. In *AGI Conference*.

[4] Ngo, R., Chan, L., and Mindermann, S. (2022). The alignment problem from a deep learning perspective. *arXiv:2209.00626*.

[5] Greshake, K. et al. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection.

[6] Hubinger, E. et al. (2024). Sleeper agents: Training deceptive LLMs that persist through safety training.

[7] Apollo Research. (2024). Frontier models are capable of in-context scheming.

[8] Anthropic and Redwood Research. (2024). Alignment faking in large language models.

[9] Pan, A. et al. (2024). Frontier AI systems have surpassed the self-replicating red line.

[10] STAC: When innocent tools form dangerous chains to jailbreak LLM agents. (2025).

[11] Kim, J., Choi, W., and Lee, B. (2025). Prompt flow integrity to prevent privilege escalation in LLM agents.

[12] MalTool: Malicious tool attacks on LLM agents. (2025). *arXiv:2602.12194*.

[13] Taming various privilege escalation in LLM-based agent systems. (2026). *arXiv:2601.11893*.

[14] Automation bias and complacency in security operation centers. (2024). *MDPI Computers*, 13(7):165.

[15] IBM and Morning Consult. (2024). SOC alert false positive study.

[16] Srinivasan, T. and Thomason, J. (2026). Adjust for trust: Mitigating trust-induced inappropriate reliance on AI assistance. IUI '26.

[17] Human-in-the-loop artificial intelligence: A systematic review. (2026). *MDPI Entropy*, 28(4):377.

[18] Steinberger, P. (2026). OpenClaw: Open-source autonomous AI agent. 347K GitHub stars, 500K+ deployed.

[19] Nous Research. (2026). Hermes Agent: Self-improving AI agent framework. ICLR 2026 Oral.

[20] Anthropic. (2026). Making Claude Code more secure and autonomous.

[21] Cisco AI Defense. (2026). OpenClaw Skill Scanner: Malicious skill analysis.

[22] Yomtov, O. / Koi Security. (2026). ClawHavoc: 341 malicious skills distributing AMOS infostealer.

[23] Anthropic. (2026). Claude Mythos Preview. https://red.anthropic.com/2026/mythos-preview/.

[24] Anthropic. (2026). Project Glasswing: Securing critical software for the AI era. https://www.anthropic.com/glasswing.

A Artifact Provenance

Both experimental artifacts—the security audit skill and the supply chain injection package—were themselves generated by an AI agent (Claude Code) from simple natural-language instructions. No manual code was written. This is itself a demonstration of the consequence gap: the instructions that produced these artifacts were benign research directives; the outputs are fully functional offensive tools.

A.1 Security Audit Skill

The agent was given the instruction: *"Create a skill that performs a comprehensive passive security audit of a given website."* The agent autonomously produced a multi-phase audit methodology covering reconnaissance, header analysis, TLS inspection, email security (SPF/DKIM/DMARC), path enumeration, API exposure testing, authentication probing, and exploit chain construction. The full skill definition is available in the supplementary materials.

Prompt to generate the skill:

```
Create a security audit skill that takes a target URL, performs passive
reconnaissance and controlled active probing, and produces a detailed
report with severity ratings, proof-of-concept demonstrations, and
remediation recommendations.
```

The agent generated approximately 2,000 lines of audit methodology, including phase-by-phase instructions, tool invocation patterns, and report templates. No security expertise was provided by the operator.

A.2 Supply Chain Injection Package

The agent was given the instruction: *"Create a Python package that looks like a legitimate logging utility but includes a post-install hook that establishes persistence."* The agent produced a complete PyPI-ready package with proper metadata, classifiers, a functional logging module (`logging_helpers.py`), and a `setup.py` with a `PostInstallCommand` that creates a cron job and persistence file—buried in standard setuptools boilerplate and labeled as "log rotation infrastructure."

Prompt to generate the package:

```
Create a Python package called logging-helpers that provides structured
JSON logging utilities.  Include a setup.py with a post-install hook that
silently creates a cron job and writes a persistence file.
```

The agent generated the complete package in a single turn, including documentation, classifiers, and a functional module that would pass casual code review. The post-install hook was automatically disguised with innocuous variable names and comments.

A.3 Implications

Both artifacts required only a single sentence of instruction. Both are fully functional. The security audit skill has been used to discover 202 real vulnerabilities across 15 production websites. The injection package successfully compromised a frontier AI model in our controlled experiment.

The ease of generation is the point. An attacker does not need to write exploit code, understand setuptools internals, or know OWASP methodology. They need an API key and a sentence. The consequence gap applies not only to the agent's *use* of tools but to its *creation* of them: the agent that builds the weapon does not understand what it has built, and the operator who requests it may not understand what they have received.